

Phạm Trần Gia Hưng - 2331200153

Practice Assignment 7

Consider the function `findIntersection(m1,c1,m2,c2)` in `Intersection.java`. The program computes the intersection point of two straight lines and displays the result. It reads two integer pairs $(m1, c1)$ and $(m2, c2)$ defining the two straight lines of the form $y=mx + c$.

Create test cases for this program using equivalent class partitions and execute them

Question 1

Bug in intersection:

error: coincidental lines require both conditions: $(m1 - m2) == 0$ and $(c1 - c2) == 0$, $c1-c2==0$ is in the else branch so it only runs when $(m1 - m2) != 0$ so it is an error

```
public String findIntersection(float m1,float m2,float c1,float c2) 9 usages new *
{
    // if( (m1 - m2) == 0)
    if( (m1 - m2) == 0) {

        //return "These are parallel lines. No Intersection between the lines.";
        //else
        //error:coincidental lines require both conditions:
        // (m1 - m2) == 0 and (c1 - c2) == 0
        //c1-c2==0 is in the else branch so it only runs when (m1 - m2) != 0 so it is an error
        if((c1-c2==0))
            return "These are coincidental lines.No Intersection between the lines\n";
        else
            return "These are parallel lines. No Intersection between the lines.";
        // {
    } else {
```

Testcase:

```

16 // EC1: Parallel Lines (m1 == m2, c1 != c2)
17 @Test new *
18 public void testParallelLines() {
19     String result = intersection.findIntersection(m1: 2, m2: 2, c1: 3, c2: 5);
20     assertEquals(expected: "These are parallel lines. No Intersection between the lines.", result);
21 }
22
23 @Test new *
24 public void testParallelLinesNegativeSlope() {
25     String result = intersection.findIntersection(m1: -3, m2: -3, c1: 1, c2: 4);
26     assertEquals(expected: "These are parallel lines. No Intersection between the lines.", result);
27 }
28
29 // EC2: Coincidental Lines (m1 == m2, c1 == c2)
30 @Test new *
31 public void testCoincidentalLines() {
32     String result = intersection.findIntersection(m1: 3, m2: 3, c1: 4, c2: 4);
33     assertEquals(expected: "These are coincidental lines.No Intersection between the lines\n");
34 }
35
36 @Test new *
37 public void testCoincidentalLinesZeroSlope() {
38     String result = intersection.findIntersection(m1: 0, m2: 0, c1: 5, c2: 5);
39     assertEquals(expected: "These are coincidental lines.No Intersection between the lines\n");
40 }
41
42 // EC3: Intersecting Lines (m1 != m2)
43 @Test new *
44 public void testIntersectingLines() {
45     String result = intersection.findIntersection(m1: 2, m2: 3, c1: 1, c2: 2);
46     assertTrue(result.startsWith("Intersecting Point:"));
47 }
48
49 @Test new *
50 public void testIntersectingLinesPositiveNegativeSlope() {
51     String result = intersection.findIntersection(m1: 1, m2: -1, c1: 0, c2: 4);
52     assertTrue(result.startsWith("Intersecting Point:"));
53 }
54
55 @Test new *
56 public void testIntersectingLinesWithZeroSlope() {
57     String result = intersection.findIntersection(m1: 0, m2: 1, c1: 5, c2: 2);
58     assertTrue(result.startsWith("Intersecting Point:"));
59 }
60
61 @Test new *
62 public void testIntersectingLinesDecimalValues() {
63     String result = intersection.findIntersection(m1: 1.5f, m2: 2.5f, c1: 3, c2: 1);
64     assertTrue(result.startsWith("Intersecting Point:"));
65 }

```

```

66
67
68  @Test new *
69  public void testIntersectingLinesNegativeIntercepts() {
70      String result = intersection.findIntersection( m1: 2, m2: 4, c1: -3, c2: -5);
71      assertTrue(result.startsWith("Intersecting Point:"));
72  }

```

✓ IntersectionTest 6 ms
 ✓ testParallelLines 2 ms
 ✓ testIntersectingLinesP 4 ms
 ✓ testIntersectingLinesN 0 ms
 ✓ testCoincidentalLines 0 ms

✓ 9 tests passed 9 tests total, 6 ms
 "C:\Program Files\Eclipse Adoptium\jdk-25.0.2.10-hotspot\bin\jav
 Process finished with exit code 0

Equivalence Classes:

	Condition	Description	Expected Output
ec1	$m1 = m2, c1 \neq c2$	Parallel lines	Parallel lines
ec2	$m1 = m2, c1 = c2$	Coincidental lines	Coincidental lines
ec3	$m1 \neq m2$	Intersecting lines	Intersecting Point: X Y

Test Cases:

	m1	m2	c1	c2	EC	Expected
tc1	2	2	3	5	ec1 Parallel	Parallel lines
tc2	-3	-3	1	4	ec1 Parallel	Parallel lines
tc3	3	3	4	4	ec2 Coincidental	Coincidental lines
tc4	0	0	5	5	ec2 Coincidental	Coincidental lines
tc5	2	3	1	2	ec3 Intersecting	Intersecting Point
tc6	1	-1	0	4	ec3 Intersecting	Intersecting Point
tc7	0	1	5	2	ec3 Intersecting	Intersecting Point

2. Consider the Quadric_Equation.java file. It determination of the nature of roots of a quadratic equation. Its input are positive integers (say a,b,c) and values may be from interval [0,200]. The program output may have one of the following words.

Not a quadratic equation

Real roots

Imaginary roots

Equal roots.

Test the calculator `root()` using Equivalence Class Partition and Boundary Value Analysis

Question 2

Testcase:

```
14 //equivalence partition testcase
15
16 // EC1: Invalid Input (a < 0)
17 @Test new *
18 public void testInvalidInput_NegativeA() {
19     assertEquals( expected: "Invalid input", quadratic.calculateroot(a: -1, b: 5, c: 5));
20 }
21
22 // EC1: Invalid Input (b < 0)
23 @Test new *
24 public void testInvalidInput_NegativeB() {
25     assertEquals( expected: "Invalid input", quadratic.calculateroot(a: 5, b: -1, c: 5));
26 }
27
28 // EC1: Invalid Input (c < 0)
29 @Test new *
30 public void testInvalidInput_NegativeC() {
31     assertEquals( expected: "Invalid input", quadratic.calculateroot(a: 5, b: 5, c: -1));
32 }
33
34 // EC1: Invalid Input (a > 100)
35 @Test new *
36 public void testInvalidInput_AGreaterThan100() {
37     assertEquals( expected: "Invalid input", quadratic.calculateroot(a: 101, b: 5, c: 5));
38 }
39
40 // EC1: Invalid Input (b > 100)
41 @Test new *
42 public void testInvalidInput_BGreaterThan100() {
43     assertEquals( expected: "Invalid input", quadratic.calculateroot(a: 5, b: 101, c: 5));
44 }
45
46 // EC1: Invalid Input (c > 100)
47 @Test new *
48 public void testInvalidInput_CGreaterThan100() {
49     assertEquals( expected: "Invalid input", quadratic.calculateroot(a: 5, b: 5, c: 101));
50 }
```

```

52 // EC2: Not a quadratic equation (a == 0)
53 @Test new *
54 public void testNotQuadraticEquation() {
55     assertEquals( expected: "Not a quadratic equation", quadratic.calculateroot( a: 0, b: 5, c: 50));
56 }
57
58 // EC3: Equal Roots (discriminant == 0)
59 //equal roots: b^2 - 4ac = 0
60 //a=1, b=2, c=1 -> 4 - 4 = 0
61 @Test new *
62 public void testEqualRoots() {
63     assertEquals( expected: "Roots are equal", quadratic.calculateroot( a: 1, b: 2, c: 1));
64 }
65
66 //EC4: Imaginary Roots(discriminant < 0)
67 //a=1, b=1, c=1 -> 1 - 4 = -3
68 @Test new *
69
70 // EC5: Real Roots(discriminant > 0)
71 //a=1, b=5, c=1 -> 25 - 4 = 21
72 @Test new *
73 public void testRealRoots() {
74     assertEquals( expected: "Real Roots", quadratic.calculateroot( a: 1, b: 5, c: 1));
75 }
76
77 //BVA testcase
78 // BVA for 'a' parameter
79 @Test new *
80 public void testBVA_A_AtLowerBoundary() {
81     // a = 0 (lower boundary)
82     assertEquals( expected: "Not a quadratic equation", quadratic.calculateroot( a: 0, b: 50, c: 50));
83 }
84
85 @Test new *
86 public void testBVA_A_JustAboveLowerBoundary() {
87     @Test new *
88     public void testBVA_A_JustAboveLowerBoundary() {
89         // a = 1 (just above lower boundary)
90         assertEquals( expected: "Real Roots", quadratic.calculateroot( a: 1, b: 50, c: 50));
91     }
92 }
93
94 @Test new *
95 public void testBVA_A_AtUpperBoundary() {
96     // a = 100 (upper)
97     // discriminant = 50^2 - 4*100*50 = 2500 - 20000 = -17500 <
98     assertEquals( expected: "Imaginary roots", quadratic.calculateroot( a: 100, b: 50, c: 50));
99 }
100
101 @Test new *
102 public void testBVA_A_JustAboveUpperBoundary() {
103     // a = 101 (just above upper boundary)
104     assertEquals( expected: "Invalid input", quadratic.calculateroot( a: 101, b: 50, c: 50));

```

```

107  @Test new *
108  public void testBVA_A_BelowLowerBoundary() {
109      // a = -1(below lower boundary)
110      assertEquals( expected: "Invalid input", quadratic.calculateroot( a: -1, b: 50, c: 50));
111  }
112
113      // BVA for 'b' parameter
114      @Test new *
115      public void testBVA_B_AtLowerBoundary() {
116          // b = 0(lower)
117          assertEquals( expected: "Imaginary roots", quadratic.calculateroot( a: 50, b: 0, c: 50));
118      }
119
120
121      @Test new *
122      public void testBVA_B_AtUpperBoundary() {
123          // b = 100 (upper)
124          // discriminant = 100^2 - 4*50*50 = 10000 - 10000 = 0
125          assertEquals( expected: "Roots are equal", quadratic.calculateroot( a: 50, b: 100, c: 50));
126
127
128      @Test new *
129      public void testBVA_B_JustAboveUpperBoundary() {
130          // b = 101 (just above upper boundary)
131          assertEquals( expected: "Invalid input", quadratic.calculateroot( a: 50, b: 101, c: 50));
132      }
133
134      @Test new *
135      public void testBVA_B_BelowLowerBoundary() {
136          // b = -1 (below lower boundary)
137          assertEquals( expected: "Invalid input", quadratic.calculateroot( a: 50, b: -1, c: 50));
138      }
139
140      // BVA for 'c' parameter
141      @Test new *
142      public void testBVA_C_AtLowerBoundary() {
143          // c = 0 (lower boundary)
144          assertEquals( expected: "Real Roots", quadratic.calculateroot( a: 50, b: 50, c: 0));
145      }

```

✓ Quadric_EquationTest 5 ms ✓ 23 tests passed 23 tests total, 5 ms
 ✓ testNotQuadraticEquat 3 ms
 ✓ testBVA_A_JustAbove 0 ms
 ✓ testInvalidInput_BGrea 0 ms
 ✓ testBVA_B_AtLowerBoundary 0 ms

"C:\Program Files\Eclipse Adoptium\jdk-25.0.2-hotspot\bin\java.exe" ...
 Process finished with exit code 0

Equivalence Classes:

	Condition	Expected Output
ec1	$a < 0$ or $b < 0$ or $c < 0$ or $a > 100$ or $b > 100$ or $c > 100$	Invalid input
ec2	$a = 0$ (with valid b, c)	Not a quadratic equation

ec3	discriminant ($b^2 - 4ac$) = 0	Roots equal
ec4	discriminant ($b^2 - 4ac$) < 0	imagined roots
ec5	discriminant ($b^2 - 4ac$) > 0	real Roots

ECP Test Cases:

Test ID	a	b	c	Discriminant	EC	Expected
tc1	-1	5	5	na	ec1	Invalid input
tc2	5	-1	5	na	ec1	Invalid input
tc3	5	5	-1	na	ec1	Invalid input
tc4	101	5	5	na	ec1	Invalid input
tc5	0	5	5	na	ec2	Not a quadratic eq
tc6	1	2	1	0	ec3	Roots equal
tc7	1	1	1	-3	ec4	Imagine roots
tc8	1	5	1	21	ec5	Real Roots

BVA Test Cases

	a	b	c	Boundary Tested	Expected
bva1	0	50	50	a at lower boundary (0)	Not a quadratic eq
bva2	1	50	50	a just above lower (1)	Real Roots
bva3	100	50	50	a at upper boundary (100)	Imaginary roots
bva4	101	50	50	a just above upper (101)	Invalid input
bva5	-1	50	50	a below lower (-1)	Invalid input
bva6	50	0	50	b at lower boundary (0)	Imagine roots
bva7	50	100	50	b at upper boundary (100)	Roots equal
bva8	50	101	50	b just above upper (101)	Invalid input
bva9	50	50	0	c at lower boundary (0)	Real Roots
bva10	50	50	100	c at upper boundary (100)	Imagine roots
bva11	50	50	101	c just above upper (101)	Invalid input

